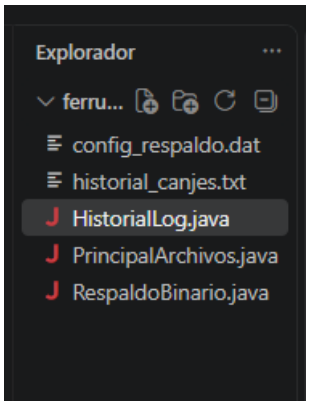
1. OBJETIVO

El objetivo de esta guía era hacer que el programa ya no solo guarde los datos en memoria, sino que los guarde de verdad en archivos en la computadora. Para eso teníamos que aprender a usar archivos de texto y archivos binarios, y a controlar los errores con try-catch para que el programa no se caiga si algo falla.

2. QUÉ HERRAMIENTAS USÉ

Usé lo mismo que en las guías anteriores: Visual Studio Code con la extensión de Java y el JDK 17. Esta vez me di cuenta que era importante ejecutar el programa desde la carpeta del proyecto, porque ahí es donde se crean solos el .txt y el .dat.



3. CÓMO ARMÉ LA ESTRUCTURA

En mi carpeta GameStoreEscolar, dentro de src, ya tenía los archivos de la guía anterior. Ahora agregué tres archivos nuevos, todos directo en src, sin meterlos en subcarpetas.

Al principio los metí en una carpeta que se llamaba HistorialLog y me salió un error rojo que decía que el paquete no coincidía. Después entendí que era porque estaban dentro de otra carpeta y no tenían puesto package. Lo que hice fue sacarlos de ahí y dejarlos directo en src, igual que los demás, y ahí sí se quitó el error.

4. CÓMO DESARROLLÉ CADA ARCHIVO

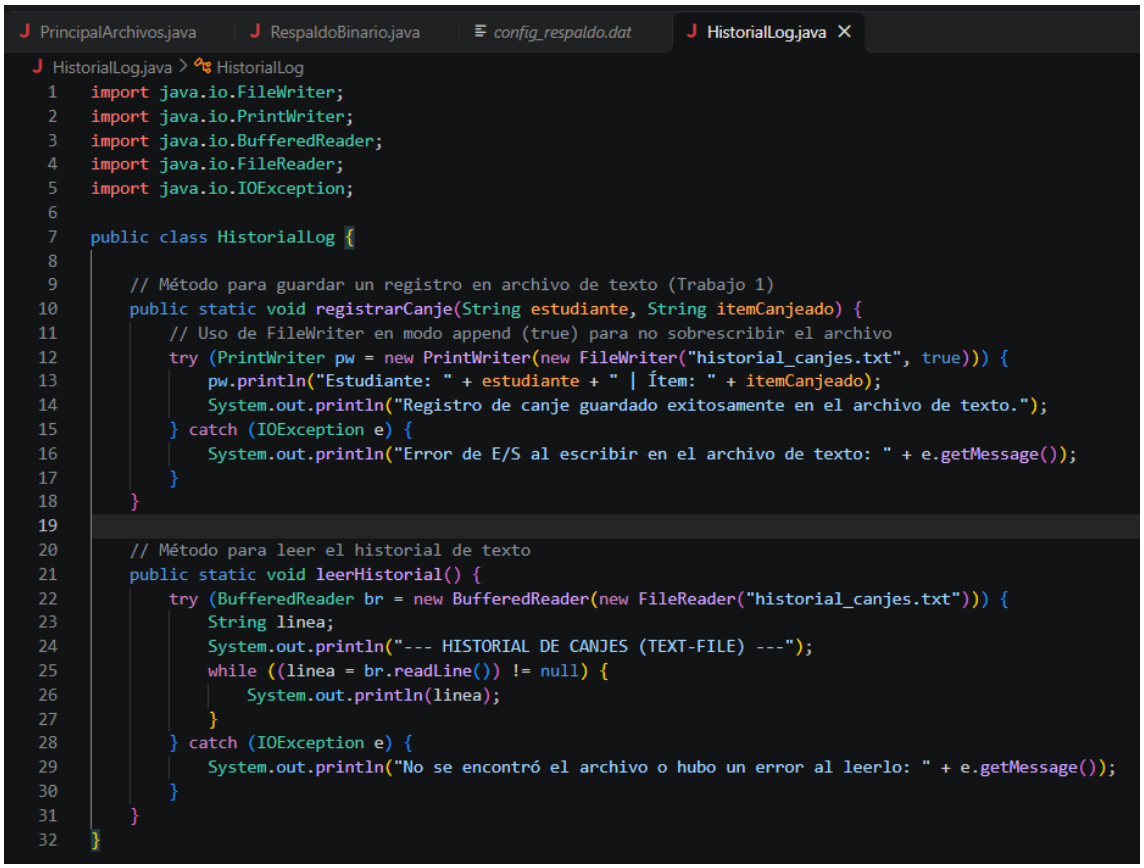
HistorialLog.java - Trabajo 1, archivo de texto

Este fue el primer archivo que hice. Su función es guardar en un archivo de texto cada vez que un estudiante hace un canje.

Para escribir usé FileWriter con PrintWriter. El FileWriter es como el tubo que llega al archivo, y el PrintWriter es como el embudo que me ayuda a escribir más fácil línea por línea. Lo puse en modo append, con un true, para que cada vez que ejecute no borre lo anterior, sino que agregue una línea nueva abajo. Por eso si lo ejecuto dos veces, el archivo queda con dos líneas.

Para leer usé FileReader con BufferedReader, que me permite leer el archivo línea por línea con un while hasta que ya no haya nada.

Todo lo puse dentro de try-catch porque si el archivo no existe o no hay permisos, Java lanza una IOException y el programa se caería. Con el catch lo que hago es mostrar un mensaje de error y que el programa siga. También usé try-with-resources, que es el try con paréntesis, para que Java cierre el archivo solo al terminar y no se quede abierto.



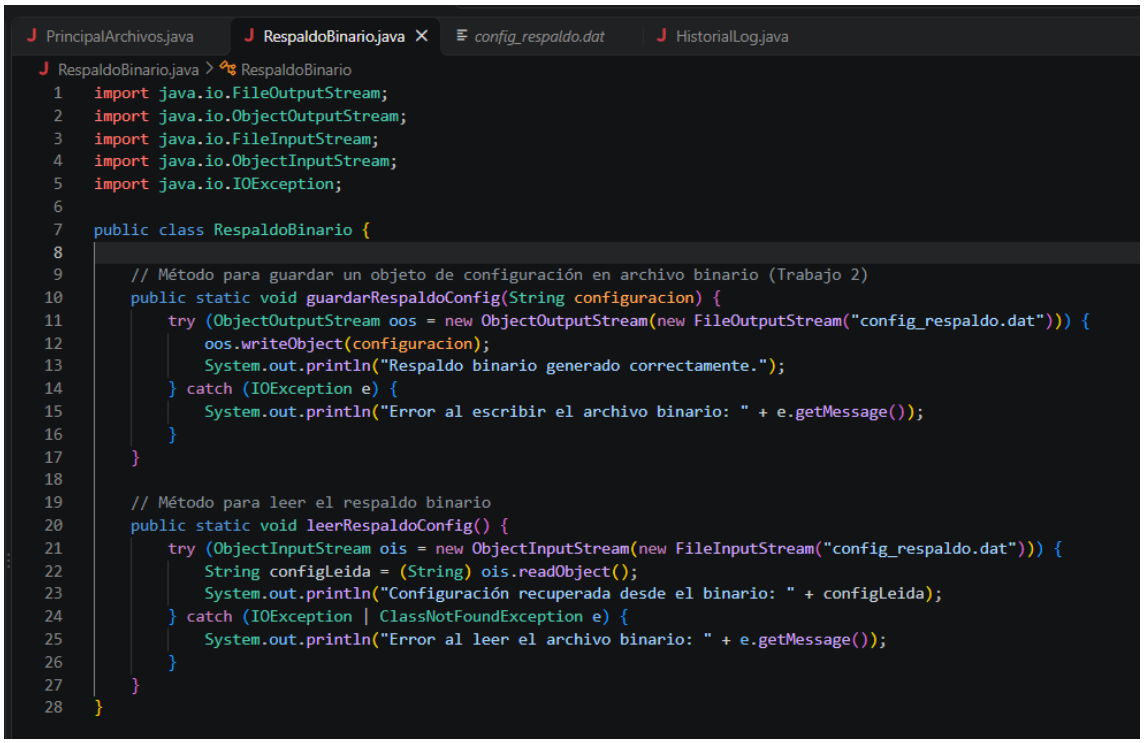
RespaldoBinario.java - Trabajo 2, archivo binario

Este segundo archivo es para guardar un respaldo de la configuración del sistema, pero en binario, no en texto. O sea que no se puede leer como un .txt normal.

Para escribir usé FileOutputStream con ObjectOutputStream. Con eso guardo el texto de la configuración como un objeto completo en un archivo .dat. Para leer hice lo contrario, con FileInputStream y ObjectInputStream, y ahí recupero el objeto y lo muestro en pantalla.

Acá también tuve que usar try-catch por lo mismo, por la IOException. Y en el de leer tuve que poner también ClassNotFoundException, porque cuando Java lee un objeto, puede ser que no encuentre la clase de ese objeto. La guía decía que había que poner los dos errores juntos con un |.

La diferencia que entendí es que el FileWriter guarda texto plano que cualquier persona puede abrir, y el ObjectOutputStream guarda en binario, que es más para respaldos y no está pensado para abrirlo como texto.



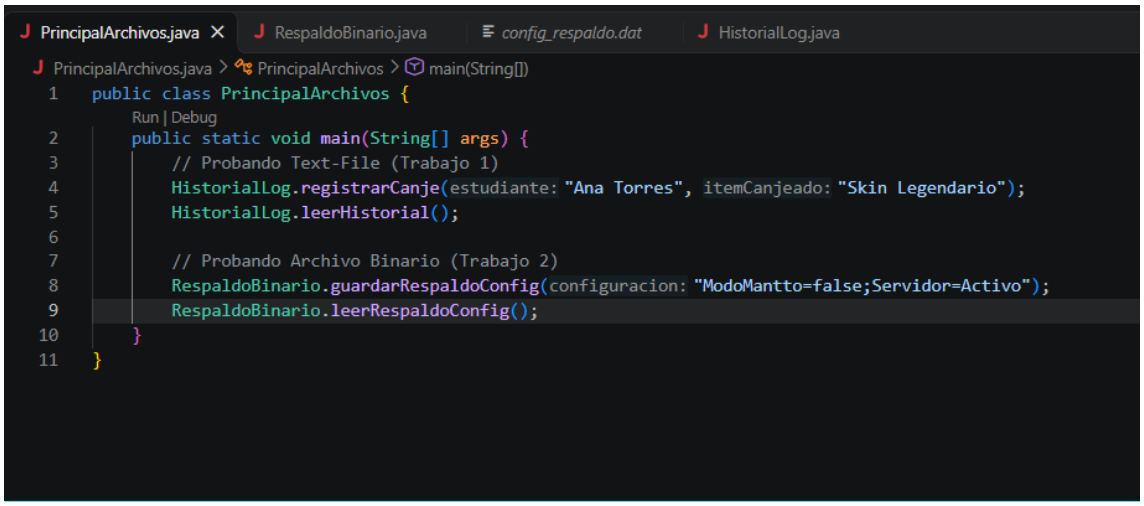
PrincipalArchivos.java - Clase de ejecución

Este es el archivo que ejecuta todo, como el PrincipalEstudiante de la guía anterior, pero para archivos. Solo tiene el main.

Adentro primero probé lo de texto: llamé a registrarCanje con el nombre Ana Torres y el ítem Skin Legendario, y después llamé a leerHistorial para ver si sí se había guardado.

Después probé lo binario: guardé una configuración que decía ModoMantto=false;Servidor=Activo, y después la leí para ver si se recuperaba igual.

Como los métodos de las otras clases son static, no tuve que crear objetos con new, solo los llamé directo con el nombre de la clase y un punto, por ejemplo HistorialLog.registrarCanje(...).

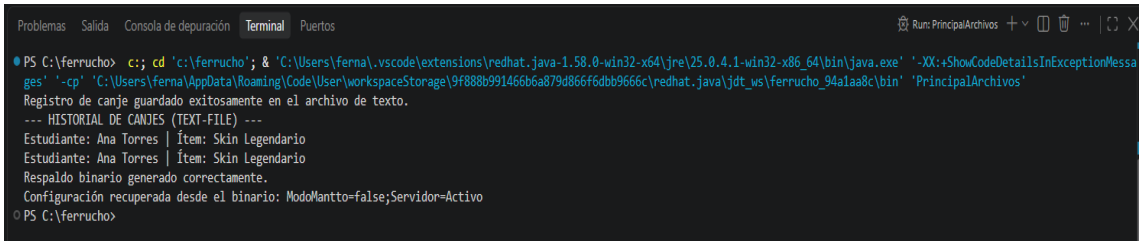


5. CÓMO LO EJECUTÉ Y QUÉ ME SALIÓ

Para ejecutarlo abrí PrincipalArchivos.java y le di clic en Run. La primera vez me salió que el registro se guardó, después me mostró el historial con la línea de Ana Torres, después me dijo que el respaldo binario se generó y al final me mostró la configuración recuperada.

Después revisé la carpeta del proyecto y ahí estaban los dos archivos nuevos: historial_canjes.txt y config_respaldo.dat. Abrí el .txt y sí se podía leer normal. El .dat no, salían símbolos raros, y ahí comprobé que sí era binario.

Si lo ejecuto otra vez, el .txt se va llenando con más líneas porque está en modo append, eso ya lo había entendido.


Problemas
Salida
Consola de depuración
Terminal
Puertos
Run: PrincipalArchivos
PS C:\ferrucho> c:; cd 'c:\ferrucho'; & 'C:\Users\ferna\.vscode\extensions\redhat.java-1.58.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\ferna\AppData\Roaming\Code\User\workspaceStorage\9f888b991466b6a879d866f6dbb9666c\redhat.java\jdt_ws\ferrucho_94a1aa8c\bin' 'PrincipalArchivos'
Registro de canje guardado exitosamente en el archivo de texto.
--- HISTORIAL DE CANJES (TEXT-FILE) ---
Estudiante: Ana Torres | Ítem: Skin Legendario
Estudiante: Ana Torres | Ítem: Skin Legendario
Respaldo binario generado correctamente.
Configuración recuperada desde el binario: ModoMantto=false;Servidor=Activo
PS C:\ferrucho>